



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR 6

Navegación completa

Planificación global (Dijkstra/A) y navegación local reactiva por campos potenciales*

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: MATLAB

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	4
2.1. Arquitectura jerárquica de navegación	4
2.2. Planificación global: Dijkstra y A*	4
2.3. Campos potenciales con componente tangencial	4
2.4. Detección de estancamiento y maniobra de escape	5
3. Diseño e implementación	6
3.1. Arquitectura del sistema	6
3.2. Datos de entrada	6
3.3. Bucle de navegación por subobjetivo	6
3.4. Detección de colisiones	7
3.5. Maniobra de escape	7
3.6. Parámetros utilizados	7
4. Resultados experimentales	8
4.1. Caso base	8
4.2. Zona conflictiva entre nodos 24 y 32	9
4.3. Diagnóstico por tramo	9
5. Discusión	10
5.1. Decisiones de diseño	10
5.2. Limitaciones	10
6. Conclusiones	11

1. Introducción

1.1. Objetivo de la práctica

El objetivo es resolver la **navegación autónoma completa** de un robot móvil sobre un mapa con obstáculos integrando, en un único programa, dos niveles funcionales: una **planificación global** sobre un grafo topológico (Dijkstra o A*) que determina la secuencia de nodos a visitar y una **navegación local reactiva** basada en campos potenciales que ejecuta cada tramo evitando obstáculos detectados por el láser.

1.2. Descripción del problema

Dado un mapa de ocupación binario $\mathcal{M}$ y un grafo topológico $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ cuyos N nodos están posicionados en el plano del mapa, junto con una matriz de costes $C \in \mathbb{R}^{N \times N}$, se desea desplazar un robot puntual desde un nodo origen s hasta un nodo destino t satisfaciendo simultáneamente:

1. **Optimalidad global:** la secuencia de nodos visitada debe corresponder a una ruta de coste mínimo en $\mathcal{G}$.
2. **Seguridad local:** en cada tramo entre dos nodos consecutivos, el robot debe evitar obstáculos de $\mathcal{M}$ usando únicamente información sensorial inmediata.
3. **Detección de fallos:** el sistema debe diagnosticar y señalar los casos en los que la navegación local no logra alcanzar un subobjetivo (mínimos locales, colisiones inevitables).

La práctica integra los algoritmos desarrollados en Lab-MR 3 (campos potenciales), Lab-MR 4 (Dijkstra) y Lab-MR 5 (A*), añadiendo dos componentes nuevos: un **campo tangencial** para mitigar mínimos locales y una **maniobra de escape** cuando el robot detecta estancamiento.

2. Fundamentos teóricos

2.1. Arquitectura jerárquica de navegación

La separación entre planificación global y navegación local es un patrón clásico en robótica móvil. El planificador global opera sobre una representación topológica abstracta y garantiza optimalidad respecto a un criterio de coste (distancia, tiempo, energía). El controlador local, en cambio, trabaja sobre la geometría real percibida por el sensor y se ocupa de la evitación de obstáculos y del seguimiento de la trayectoria de referencia.

Esta separación confiere robustez (un fallo local no requiere replanificación global) y permite reutilizar componentes desarrollados en prácticas previas como bloques independientes.

2.2. Planificación global: Dijkstra y A*

Para la capa global se reutilizan las funciones desarrolladas en Lab-MR 4 y Lab-MR 5:

- `dijkstra(C, s, t)`: ruta de coste mínimo sobre la matriz de costes original.
- `astar(C_eucl, XY, s, t)`: A* con heurística euclídea sobre la matriz C^{eucl} construida a partir de las coordenadas de los nodos.

Como la heurística euclídea solo es consistente cuando los pesos coinciden con la métrica euclídea, A* se aplica sobre la matriz transformada

$$C_{ij}^{\text{eucl}} = \begin{cases} \|p_i - p_j\|_2 & \text{si existe arco } (i, j), \\ 0 & \text{en caso contrario,} \end{cases} \quad (1)$$

manteniendo la topología de $\mathcal{G}$ pero con costes geométricos. La salida de ambos planificadores es la misma: un coste total y una secuencia de índices de nodos que define los **subobjetivos** para la capa local.

2.3. Campos potenciales con componente tangencial

Sobre la base teórica de Lab-MR 3 (campo atractivo F_{att} y repulsivo F_{rep} que diverge en $d \rightarrow 0$ y se anula en $d = D$), esta práctica añade un **campo tangencial** cuyo propósito es facilitar el bordeado de obstáculos cuando atracción y repulsión están alineadas.

Sea q_{near} el obstáculo más cercano dentro del rango de influencia y $\mathbf{v}_{\text{obs}} = q_{\text{near}} - q$ el vector del robot al obstáculo. La dirección tangencial se elige perpendicular a $\mathbf{v}_{\text{obs}}$ y orientada hacia el lado en el que se encuentra el subobjetivo, mediante el signo del producto vectorial entre el vector unitario al objetivo $\hat{\mathbf{v}}_{\text{goal}}$ y $\mathbf{v}_{\text{obs}}$:

$$s = \text{sign}(\hat{v}_{\text{goal},x} v_{\text{obs},y} - \hat{v}_{\text{goal},y} v_{\text{obs},x}) \quad (2)$$

$$\hat{\mathbf{t}} = \begin{cases} (-\hat{v}_{\text{obs},y}, \hat{v}_{\text{obs},x}) & \text{si } s > 0, \\ (\hat{v}_{\text{obs},y}, -\hat{v}_{\text{obs},x}) & \text{si } s \leq 0. \end{cases} \quad (3)$$

La magnitud se modula por la cercanía al obstáculo:

$$\mathbf{F}_{\text{tan}} = \gamma_{\text{tan}} \max\left(0, 1 - \frac{d_{\text{near}}}{D}\right) \hat{\mathbf{t}} \quad (4)$$

La fuerza resultante es la suma vectorial:

$$\mathbf{F}_{\text{res}} = \mathbf{F}_{\text{att}} + \mathbf{F}_{\text{rep}} + \mathbf{F}_{\text{tan}} \quad (5)$$

2.4. Detección de estancamiento y maniobra de escape

Aunque el campo tangencial reduce la frecuencia de mínimos locales, no los elimina por completo. Para los casos residuales se introduce un mecanismo de detección basado en el **progreso** reciente: se mantiene una ventana deslizante con las últimas W distancias al subobjetivo, y se considera estancamiento cuando el progreso acumulado $\Delta d = d_{\text{ini}} - d_{\text{fin}}$ es inferior a un umbral Δ_{min} .

Al detectar estancamiento se ejecuta una **maniobra de escape**: se prueban K ángulos uniformemente distribuidos alrededor de la dirección hacia el subobjetivo, se descartan los que conducen a colisión y se selecciona el que maximiza una puntuación combinada de **despeje** (mediana de las distancias láser tras el desplazamiento) y **cercanía al subobjetivo**:

$$\text{score}(a) = 0.7 \bar{d}_{\text{lidar}}(a) - 0.3 \|q_{\text{sub}} - q(a)\| \quad (6)$$

Si todas las direcciones conducen a colisión, el subobjetivo se declara inalcanzable y la navegación termina con fallo controlado. Tras un número máximo de eventos de estancamiento, se diagnostica mínimo local y se aborta el tramo.

3. Diseño e implementación

3.1. Arquitectura del sistema

La solución se implementa en un único script de MATLAB (`navegacion_autonoma_p7.m`) organizado en cinco fases secuenciales:

1. Carga del mapa de ocupación y del grafo topológico.
2. Selección interactiva del planificador, origen y destino.
3. Planificación global (Dijkstra o A* sobre C^{eucl}).
4. Bucle de navegación local por subobjetivos.
5. Visualización de resultados y log de tramos.

Las funciones críticas (`calcular_campos`, `maniobra_escape`, `hay_colision`, `construir_costes_eucl`) se definen como funciones locales en el mismo script, y la planificación global se delega en las funciones `dijkstra.m` y `astar.m` desarrolladas en prácticas anteriores.

3.2. Datos de entrada

Se utilizan los ficheros proporcionados por el enunciado:

- `mapa2.pgm`: imagen binaria del mapa de ocupación.
- `mapa2.m`: script que define las variables `nodos` (matriz $N \times 3$ con $[id, x, y]$) y `costes` (matriz $N \times N$ de adyacencia ponderada).

El mapa se convierte en un `binaryOccupancyMap` y el grafo se carga ejecutando el script con `run()`. Las dimensiones se validan antes del bucle principal.

3.3. Bucle de navegación por subobjetivo

La función `navegar_hasta_subobjetivo` ejecuta un bucle iterativo de hasta `max_iter_seg = 1200` pasos. En cada iteración:

1. Comprueba si la distancia al subobjetivo es inferior a la tolerancia `goal_tol`. Si es así, retorna con éxito.
2. Lee el láser simulado mediante `rayIntersection` sobre el mapa.
3. Calcula los tres campos ((5)) y la fuerza resultante.

4. Si $\|\mathbf{F}_{\text{res}}\| < \text{min_force}$, lanza la maniobra de escape.
5. Calcula la pose candidata avanzando un paso de longitud $\min(v, d_{\text{goal}})$ en la dirección de $\mathbf{F}_{\text{res}}$.
6. Verifica colisión en la pose candidata; si la hay, lanza maniobra de escape.
7. Actualiza el historial de progreso. Si la ventana de W pasos no acumula progreso suficiente, incrementa el contador de estancamientos y ejecuta una maniobra de escape correctora.
8. Si el contador alcanza max_stuck_events , declara mínimo local y aborta.

3.4. Detección de colisiones

La función `hay_colision` no comprueba un único punto sino un **esténcil de 9 puntos** (centro, 4 cardinales y 4 diagonales) a distancia `collision_margin`. Esto modela implícitamente un radio de seguridad del robot y previene colisiones por aproximación a esquinas.

3.5. Maniobra de escape

La función `maniobra_escape` prueba $K = 16$ direcciones equiespaciadas alrededor de la dirección al subobjetivo, descarta las que colisionan, y de las restantes selecciona la de mayor puntuación según la ecuación (6). Si ninguna dirección es viable, retorna fallo y el bucle local lo propaga como motivo del aborto.

3.6. Parámetros utilizados

La Tabla 1 recoge los valores empleados, calibrados empíricamente sobre `mapa2.pgm`.

Tabla 1: Parámetros del controlador integrado de navegación autónoma.

Parámetro	Valor	Descripción
v	0.45	Paso lineal por iteración.
D	6.5	Rango de influencia repulsiva.
α	1.4	Coeficiente del campo atractivo.
β	200	Coeficiente del campo repulsivo.
γ_{tan}	0.45	Coeficiente del campo tangencial.
r_{max}	12	Alcance máximo del láser.
rango angular	$[-\pi/2, \pi/2]$	Sector explorado por el láser.
goal_tol	0.75	Tolerancia de llegada a subobjetivo.
collision_margin	0.60	Radio del estencil de colisión.
W	40	Tamaño de la ventana de progreso.
Δ_{min}	0.40	Progreso mínimo en la ventana.
max_stuck_events	4	Estancamientos antes de declarar mínimo local.
escape_step	1.00	Longitud del paso de escape.
K	16	Direcciones probadas en la maniobra de escape.
iter. máx. por tramo	1200	Límite por subobjetivo.

4. Resultados experimentales

4.1. Caso base

Se ha evaluado el sistema sobre el mapa `mapa2.pgm` con el grafo `mapa2.m`, eligiendo como destino por defecto el nodo de la zona superior derecha (máxima coordenada $x + y$). Tanto Dijkstra como A* producen rutas coincidentes en coste sobre C^{eucl} , y la fase local recorre los subobjetivos sin incidentes en la mayoría de las trayectorias.

[PENDIENTE: Captura sobre `mapa2.pgm` con los nodos del grafo dibujados, la ruta global de Dijkstra (o A*) en azul/amarillo y la trayectoria local roja del robot superpuesta. Caso típico que llega al destino.]

Figura 1: Caso base: planificación global Dijkstra/A* + navegación local por campos potenciales sobre `mapa2.pgm`.

4.2. Zona conflictiva entre nodos 24 y 32

El enunciado destaca la región entre los nodos 24 y 32 como caso de estudio para los mínimos locales. En esta zona el pasillo es estrecho y los obstáculos están dispuestos de forma aproximadamente simétrica respecto al eje del robot, lo que genera tres modos de fallo:

- equilibrio entre F_{att} y F_{rep} , con $\|F_{res}\| \approx 0$,
- oscilaciones laterales entre las dos paredes,
- proximidad simultánea a obstáculos por ambos lados que activa el estencil de colisión.

Con la versión básica (sin F_{tan} ni maniobra de escape), el robot se queda atascado de forma reproducible en esta zona. Con la versión mejorada, el campo tangencial empuja al robot hacia uno de los dos lados y, si aún así se detecta estancamiento, la maniobra de escape selecciona una dirección con mayor despeje. La diferencia es cualitativamente clara.

[PENDIENTE: Dos subpaneles del mismo origen-destino que atraviesan la zona 24–32: izquierda sin mejora (campos potenciales puros, trayectoria atascada u oscilante); derecha con mejora (campo tangencial + maniobra de escape, trayectoria que llega al destino). Anotar iteraciones y éxito/fallo.]

Figura 2: Comparativa en la zona conflictiva 24–32: navegación básica frente a navegación con mejora anti-mínimos locales.

4.3. Diagnóstico por tramo

El script genera un **log** con el motivo de finalización de cada tramo (`ok`, `minimo_local`, `colision_sin_escape`, `fuerza_nula_sin_escape`, `max_iter`). Esta traza facilita identificar qué subobjetivo es problemático y orienta la calibración posterior de los parámetros.

5. Discusión

5.1. Decisiones de diseño

Elección del campo tangencial frente a perturbación aleatoria. Una alternativa clásica para evitar mínimos locales es añadir ruido aleatorio a la fuerza resultante. El campo tangencial es preferible porque es **determinista** (reproducible entre ejecuciones) y está **geométricamente motivado**: orienta el escape hacia el lado donde se encuentra el subobjetivo, no en una dirección arbitraria.

Doble red de seguridad. La detección de colisión por estencil de 9 puntos y la verificación previa al avance forman dos líneas de defensa independientes. Aunque el campo repulsivo debería ser suficiente en teoría, en la práctica las discretizaciones del paso lineal y del láser pueden producir aproximaciones a esquinas que el campo continuo no detecta a tiempo.

Detección de estancamiento por progreso, no por velocidad. Medir el estancamiento como falta de **progreso** hacia el subobjetivo (no como velocidad nula) es más robusto: el robot puede estar moviéndose rápido en oscilaciones laterales sin acercarse al objetivo, y la métrica de progreso lo detecta correctamente.

5.2. Limitaciones

- El **campo tangencial** puede introducir **rodeos innecesarios** en pasillos amplios donde el método básico ya funciona. La modulación por $1 - d_{\text{near}}/D$ atenua este efecto pero no lo elimina.
- La **maniobra de escape** es local: no almacena memoria de zonas problemáticas, por lo que el robot puede volver a entrar en la misma trampa si el subobjetivo lo arrastra.
- El **paso lineal fijo** v no se adapta a la cercanía de obstáculos. En zonas estrechas un paso adaptativo reduciría el riesgo de colisión.
- La **planificación global no contempla obstáculos dinámicos** ni cierres de paso: si una arista del grafo es transitable en teoría pero el pasillo correspondiente está bloqueado, el sistema no replanifica.
- El láser simulado tiene rango angular $[-\pi/2, \pi/2]$, por lo que los obstáculos por detrás del robot son invisibles. Esto es coherente con el enfoque reactivo pero impide reculadas seguras.

6. Conclusiones

Se ha desarrollado un sistema integrado de navegación autónoma que combina planificación global y control reactivo en un único programa MATLAB, reutilizando como bloques los algoritmos desarrollados en prácticas anteriores (Dijkstra, A* y campos potenciales). Las contribuciones específicas de esta práctica son:

1. **Integración jerárquica:** la ruta del planificador global actúa como secuencia de subobjetivos para la capa local, con diagnóstico independiente por tramo.
2. **Campo tangencial** para mitigar mínimos locales sin recurrir a aleatoriedad, orientado según el producto vectorial entre la dirección al subobjetivo y la dirección al obstáculo más cercano.
3. **Detección de estancamiento** mediante ventana deslizante de progreso, más robusta que la detección por velocidad nula.
4. **Maniobra de escape** con selección basada en una puntuación combinada de despeje y cercanía al subobjetivo, evaluada sobre K direcciones candidatas.
5. **Selección entre Dijkstra y A*** en tiempo de ejecución, con construcción automática de la matriz C^{eucl} para garantizar consistencia de la heurística.

Las pruebas en la zona conflictiva 24–32 confirman que la combinación campo tangencial + maniobra de escape es eficaz para superar configuraciones que el método básico no resuelve. Como trabajo futuro se plantea la incorporación de un planificador local más elaborado (Dynamic Window Approach), un paso lineal adaptativo a la cercanía de obstáculos, y un mecanismo de replanificación global ante bloqueos persistentes detectados por la capa local.